國立臺北科技大學
National Taipei University of Technology

資料結構
Data Structure

Lab 08

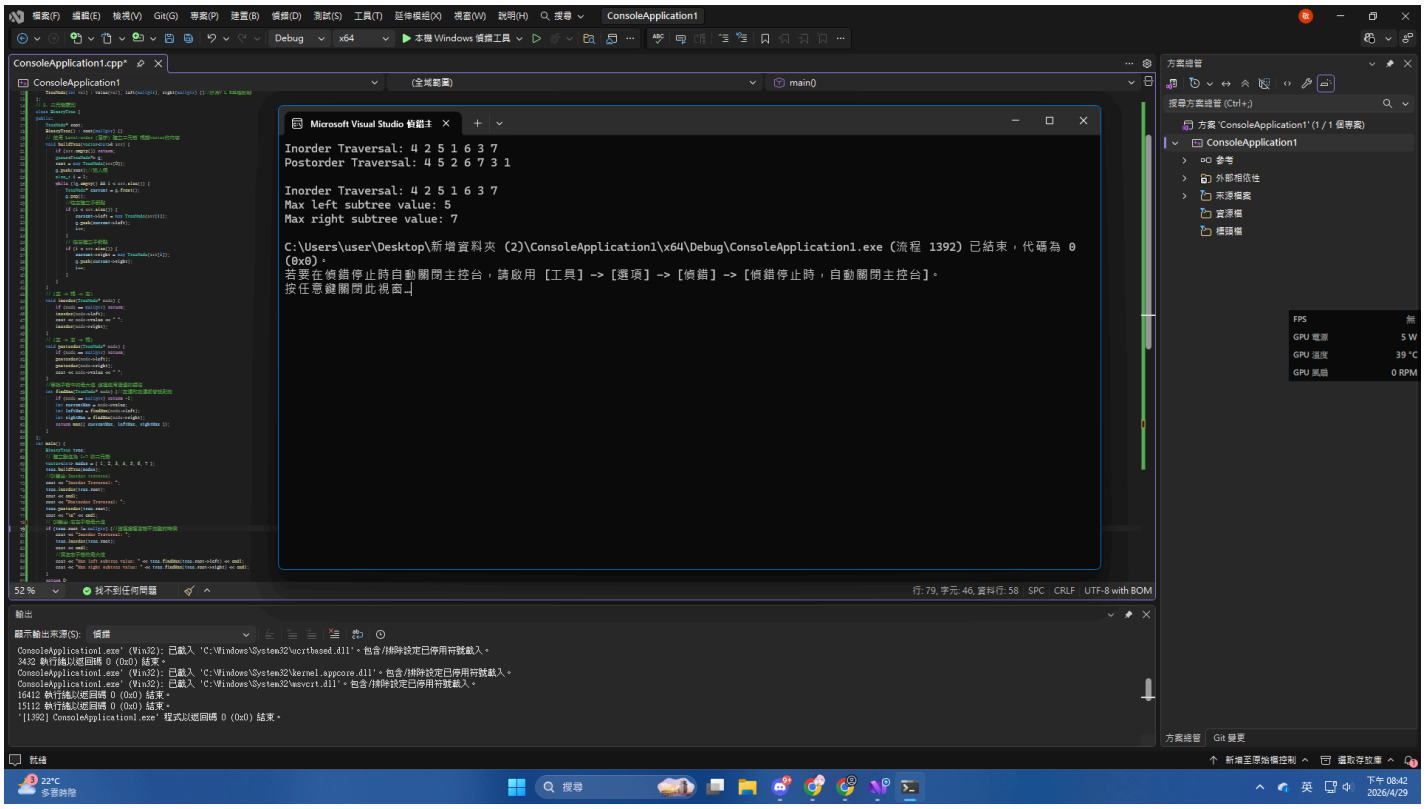
姓名: 盧敬恩
學號: 114310123

Lab08-Ex1. (Tree)
完成Postorder Traversal
Code

```
#include <iostream>
#include <vector>
#include <queue>
#include <algorithm>
using namespace std;
//定義樹類別
class TreeNode {
public:
    int value;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int val) : value(val), left(nullptr), right(nullptr) {}//分為V L R三種節點
};
// 2. 二元樹類別
class BinaryTree {
public:
    TreeNode* root;
    BinaryTree() : root(nullptr) {}
    // 使用 Level-order (層序) 建立二元樹 根據vector的內容
    void buildTree(vector<int>& arr) {
        if (arr.empty()) return;
        queue<TreeNode*> q;
        root = new TreeNode(arr[0]);
        q.push(root);//放入根
        size_t i = 1;
        while (!q.empty() && i < arr.size()) {
            TreeNode* current = q.front();
            q.pop();
            //往左建立子節點
            if (i < arr.size()) {
                current->left = new TreeNode(arr[i]);
                q.push(current->left);
                i++;
            }
            // 往右建立子節點
            if (i < arr.size()) {
                current->right = new TreeNode(arr[i]);
                q.push(current->right);
                i++;
            }
        }
    }
```

```
 // (左 -> 根 -> 右)
    void inorder(TreeNode* node) {
        if (node == nullptr) return;
        inorder(node->left);
        cout << node->value << " ";
        inorder(node->right);
    }
    // (左 -> 右 -> 根)
    void postorder(TreeNode* node) {
        if (node == nullptr) return;
        postorder(node->left);
        postorder(node->right);
        cout << node->value << " ";
    }
    //尋找子樹中的最大值 這裡使用遞迴的語法
    int findMax(TreeNode* node) {//左邊和右邊都會找到完
        if (node == nullptr) return -1;
        int currentMax = node->value;
        int leftMax = findMax(node->left);
        int rightMax = findMax(node->right);
        return max({ currentMax, leftMax, rightMax });
    }
};
int main() {
    BinaryTree tree;
    // 建立數值為 1~7 的二元樹
    vector<int> nodes = { 1, 2, 3, 4, 5, 6, 7 };
    tree.buildTree(nodes);
    //Q1輸出:Inorder traversal
    cout << "Inorder Traversal: ";
    tree.inorder(tree.root);
    cout << endl;
    cout << "Postorder Traversal: ";
    tree.postorder(tree.root);
    cout << "\n" << endl;
    // Q2輸出:右左子樹最大值
    if (tree.root != nullptr) {//這裡處理當樹不完整的時候
        cout << "Inorder Traversal: ";
        tree.inorder(tree.root);
        cout << endl;
        //算左右子樹的最大值
        cout << "Max left subtree value: " << tree.findMax(tree.root->left) << endl;
        cout << "Max right subtree value: " << tree.findMax(tree.root->right) << endl;
    }
    return 0;
}
```

Result



Discussion&bonus

(1) 說明你們怎麼在左右子樹中找到最大值:

```
max({ currentMax, leftMax, rightMax })
```

看當前節點的值和左右子樹的最大值(透過遞迴)

```
int findMax(TreeNode* node) {//左邊和右邊都會找到完
    if (node == nullptr) return -1;
    int currentMax = node->value;
    int leftMax = findMax(node->left);
    int rightMax = findMax(node->right);
    return max({ currentMax, leftMax, rightMax });
}
```

透過過遞，直到node == null ptr也就是子樹的末葉時，才會回傳

如果遇到左右子樹不為完整樹,會發生甚麼事?

也就是子葉僅存在左分支或是右分枝。這時就會停止往下一步繼續找最大值，因為
`if (node == nullptr) return -1`所以程式碼不會報錯誤。
當找對大值回傳-1絕對不會成為最大值。